Ascend C算子编译与调试调优能力概述

作者 汪凯

时间 2026/03

课程背景与目标

课程背景:

随着昇腾AI处理器架构的持续演进，下一代AI处理器架构在硬件设计对CUBE单元做修改，VECTOR单元支持两种工作模式： SIMD和SIMT，以VF(Vector Function）为粒度切换，这些硬件层面的变化直接影响算子编程编译和调试调优的方式。

课程目标:

- 掌握Ascend C算子编程编译及图框架接入方式方法
- 掌握下一代芯片的Ascend C 调试调优接口的基本使用
- 熟悉下一代芯片的Ascend C算子的调试调优工具的能力

前置知识:

- 熟悉AscendC编程模型、下一代芯片架构、算子工具的使用

CANN

目录

Part 1 Ascend C 算子开发流程概述

Part 2 单算子、入图开发方式介绍

Part 3 调测接口使用方法及案例

Part 4 配套工具的使用方法及案例

Ascend C 算子开发流程概述

- Kernel代码编写
- <<<>>>调用

- 命令行编译
- Cmake工程项目

- 支持调测接口**printf/dump**等
- 支持孪生调试
- 支持二进制文件解析
- 支持上板调试**msDebug**
- 支持正确性校验**msSanitizer**

- Profling性能数据采集，如MAC、带宽利用率等
- Roofline瓶颈分析
- 指令流水图
- Pipe流水图
- SIMT VF 指令 Stall 分析

阶段	工具	使用场景
调试	msDebug	**上板调试**：可以设置断点、打印变量和内存、进行单步调试、中断运行（coredump后支持堆栈能力正在构建）
	msSantizer	**异常检测工：**包含内存检测、竞争检测、未初始化和同步检测
	msobjdump	**二进制解析**：解析算子ELF文件(Executable and Linkable Format），方便开发者直观获得kernel文件信息
调优	msProf	**性能采集和优化**：采集和分析运行在昇腾AI处理器上算子的关键性能指标

CANN

目录

Part 1 算子开发调试调优总览概述

Part 2 单算子、接入图框架开发介绍

Part 3 调测接口使用方法及案例

Part 4 配套工具的使用方法及案例

Ascend C 算子三种编程范式回顾

SIMD编程范式：

```
// main.asc
#include "kernel_operator.h"
#include "acl/acl.h"

__global__ __vector__ void hello_world()
{
    AscendC::printf("Hello World!!!\n");
}

int32_t main(int argc, char const *argv[])
{
    aclInit(nullptr);
    int32_t deviceId = 0;
    aclrtSetDevice(deviceId);
    aclrtStream stream = nullptr;
    aclrtCreateStream(&stream);

    constexpr uint32_t numBlocks = 8;
    hello_world<<<numBlocks, nullptr, stream>>>();
    aclrtSynchronizeStream(stream);

    aclrtDestroyStream(stream);
    aclrtResetDevice(deviceId);
    aclFinalize();
    return 0;
}
```

SIMD VF/SIMT VF混合编程范式：

```
__simd_vf__ inline void simd_compute(...) { ... }

__simt_vf__ __launch_bounds__(THREAD_COUNT) inline
void simt_compute(...)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    ...
}

__global__ __aicore__ void custom_op_kernel(...)
{
    __ubuf__ float input_buf[INPUT_SIZE];
    asc_copy_gm2ub_align(...);
    ...
    asc_vf_call<simd_compute>(...);
    ...
    asc_vf_call<simt_compute>(dim3(THREAD_COUNT), ...);
     ...
}

custom_op_kernel<<<num_blocks, dyn_ubuf_size,
stream>>>(...);
```

SIMT 编程范式：

```
__global__ __launch_bounds__(MAX_THREAD_COUNT)
void gather_custom(type_data* input,
    type_idx* index, type_data* gather_output,)
{
    int32_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    .......
        printf("gather_custom: thread idx: %d.\n", idx);
    ......
}

gather_custom<<<1, 1, dyn_ubuf_size,
stream>>>(input,   index, gather_output);
```

SIMD关键字	使用说明
__cube__	标识该核函数仅在Cube核执行
__vector__	标识该核函数仅在Vector核执行
__mix__(aic, aiv)	标识该核函数同时在Cube核和Vector核上执行

https://gitcode.com/cann/asc-devkit/tree/master/examples

CANN

ASC支持异构编译与<<<>>>直调，提供与业界一致编程编译体验

- 对标业界，支持异构融合编译和<<<>>>直调；
- 通过后缀名“.asc”或 编译选项“-x asc”使能异构编译；

两种场景编译方式：

1 即时验证：使用命令行编译，示例：

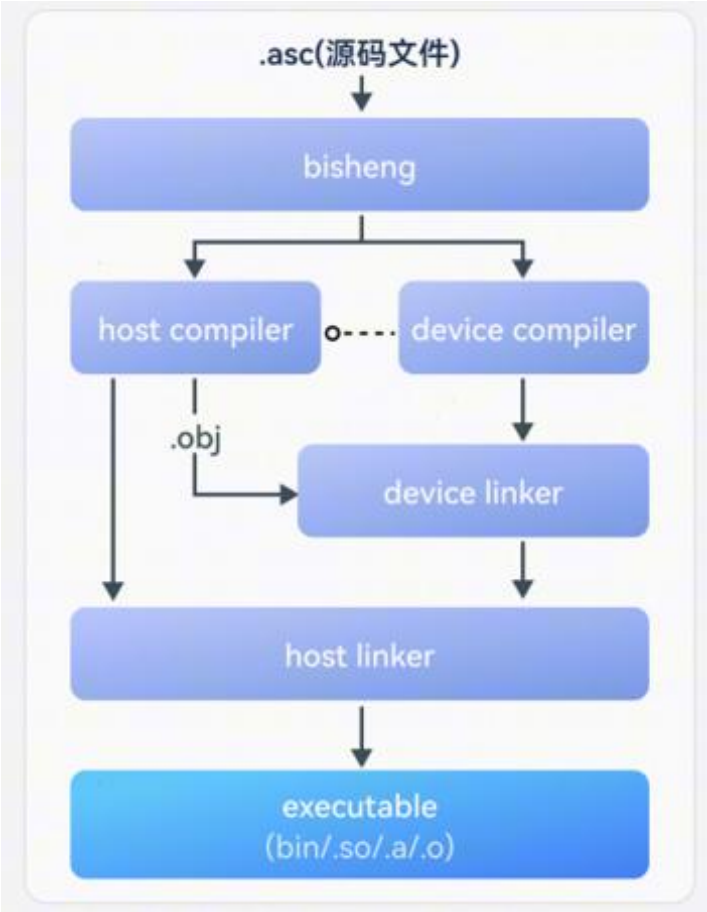
```
bisheng main.asc --npu-arch dav-3510 -o main
```

2 工程集成：使用标准CMake编译，示例：

```
cmake_minimum_required(VERSION 3.16)

find_package(ASC REQUIRED)
project(kernel_samples LANGUAGES ASC CXX)

add_executable(main
    main.asc
)

target_compile_options(main PRIVATE
    $<$<COMPILE_LANGUAGE:ASC>:--npu-arch=dav-3510>
)
```

```
mkdir build; cd build; cmake ../;make
```

异构编译原理：



Cmake工程样例

https://gitcode.com/cann/asc-devkit/blob/master/examples/00_introduction/00_helloworld/hello_world_npu/CMakeLists.txt

CANN

Ascend C算子接入Pytorch框架开发方式

1

```
__global__ __vector__ void add_custom(GM_ADDR x, GM_ADDR y, GM_ADDR z, uint32_t totalLength)
{
    KernelAdd op;
    op.Init(x, y, z, totalLength);
    op.Process();
}
```

2

```
namespace ascendc_ops {
at::Tensor ascendc_add(const at::Tensor& x, const at::Tensor& y)
{
    auto aclStream = c10_npu::getCurrentNPUStream().stream(false);
    at::Tensor z = at::empty_like(x);
    uint32_t blockDim = 8;
    uint32_t totalLength = 1;
    for (uint32_t size : x.sizes()) {
        totalLength *= size;
    }
    add_custom<<<blockDim, nullptr, aclStream>>>((uint8_t*)(x.mutable_data_ptr()), (uint8_t*)(y.mutable_data_ptr()), (uint8_t*)(z.mutable_data_ptr()), totalLength);
    return z;
}
} // namespace ascendc_ops
```

PyTorch提供TORCH_LIBRARY宏作为自定义算子注册的核心接口，用于创建并初始化自定义算子库，注册后在Python侧可以通过torch.ops.namespace.op_name方式进行调用

3

```
TORCH_LIBRARY(ascendc_ops, m)
{
    m.def("ascendc_add(Tensor x, Tensor y) -> Tensor");
}
```

TORCH_LIBRARY_IMPL用于将算子逻辑绑定到特定的DispatchKey（PyTorch设备调度标识）。针对NPU设备，需要将算子实现注册到PrivateUse1这一专属的DispatchKey上

4

```
TORCH_LIBRARY_IMPL(ascendc_ops, PrivateUse1, m)
{
    m.impl("ascendc_add", TORCH_FN(ascendc_ops::ascendc_add));
}
```

自定义算子接入Pytorch框架工程编译与验证

5

```
cmake_minimum_required(VERSION 3.16)
find_package(ASC REQUIRED)
......
add_library(ascendc_ops SHARED
    add_custom.asc
)

target_link_libraries(ascendc_ops PRIVATE
    torch_npu
    Python3::Python
)

target_compile_options(ascendc_ops PRIVATE
    ${TORCH_CXX_FLAGS}
    $<$<COMPILE_LANGUAGE:ASC>:--npu-arch=dav-3510>
)
```

6

在add_custom_test.py调用脚本中，通过torch.ops.load_library加载生成的自定义算子库，调用注册的ascendc_add函数，并通过对比NPU输出与CPU标准加法结果来验证自定义算子的数值正确性

```
import sys
import os
import torch
import torch_npu
from torch_npu.testing.testcase import TestCase, run_tests
torch.ops.load_library("libascendc_ops.so")


class TestCustomAdd(TestCase):
    def test_add_custom_ops(self):
        length = [8, 2048]
        x = torch.rand(length, device='cpu', dtype=torch.float16)
        y = torch.rand(length, device='cpu', dtype=torch.float16)
        output = torch.ops.ascendc_ops.ascendc_add(x.npu(), y.npu()).cpu()
        cpuout = torch.add(x, y)
        self.assertRtolEqual(output, cpuout)


if __name__ == "__main__":
    run_tests()
```

算子样例代码：
https://gitcode.com/cann/asc-devkit/tree/master/examples/02_features/01_triple_chevron_notation/torch_library

https://gitcode.com/cann

CANN

本章小结

- **介绍Ascend C算子编程及编译**
  - ✓ Ascend C单算子编程编译
  - ✓ Pytorch框架接入的编程编译

目录

Part 1 算子开发调试调优总览概述

Part 2 单算子、入图开发方式介绍

Part 3 调测接口使用方法及案例

Part 4 配套工具的使用方法及案例

AICore 调试调优接口 —— printf/assert/dump /time stamp

printf样例代码片段:

```
__global__ __aicore__ void __kernel () {
{
 ……
 printf("Hello World!!!\n");
 ……
}
__kernel<<<4, nullptr, stream>>>();
```

Printf打印示例:

```
(base) root@huawei:/home/[redacted]# ./main
Hello World!!!
Hello World!!!
Hello World!!!
Hello World!!!
```

assert样例代码片段:

```
__global__ __aicore__ void __kernel (bool isTrasnA) {
{
 ……
 assert(isTransA == true)
 ……
}
__kernel<<<4, nullptr, stream>>>(false);
```

assert打印示例:

```
(base) root@huawei:/home/[redacted]# ./assert
[ASSERT] assert.asc:6: Assertion `isTransA == true'
[ASSERT] assert.asc:6: Assertion `isTransA == true'
[ASSERT] assert.asc:6: Assertion `isTransA == true'
[ASSERT] assert.asc:6: Assertion `isTransA == true'
```

dump样例代码片段:

```
__global__ __aicore__ void __kernel (__gm__ float * x, __gm__
float * y) {
{
 ……
 asc_dump_gm(x, 5, 32);
 ……
}
```

打印示例如下:

```
DumpTensor: desc=5, addr=0, data_type=float16, position=UB, dump_size=32
[19.000000, 4.000000, 38.000000, 50.000000, 39.000000, 67.000000, 84.000000, 98.000000, 21.000000, 36.000000,
18.000000, 46.000000, 10.000000, 92.000000, 26.000000, 38.000000, 39.000000, 9.000000, 82.000000, 37.000000,
35.000000, 65.000000, 97.000000, 59.000000, 89.000000, 63.000000, 70.000000, 57.000000, 35.000000, 3.000000,
16.000000,
42.000000]
```

timestamp样例代码片段:

```
__global__ __aicore__ void __kernel (__gm__ float * x, __gm__
float * y) {
{
 ……
 asc_time_stamp(65577);
 ……
}
```

打印结果如下（Dump信息头等仅在使用自定义算子工程时才会打印）：

```
...// 一些框架内部的打点信息
descId is 65577, rsv is 0, timeStamp is 13806084506158, pcPtr is 20619064414544.
```

CANN

SIMD VF 调试调优接口 —— printf/assert/dump

1 必须使用asc_vf_call来调用VF函数

```
__global__ __aicore__ void __kernel (__gm__ float * x, __gm__ float * y) {
{
  ......
  __ubuf__ float* src0_addr = (__ubuf__ float*)srcLocalTenosr.GetPhyAddr();
  __ubuf__ float* dst_addr = (__ubuf__ float*)dstLocalTenosr.GetPhyAddr();
  asc_vf_call<add_vf>(tmp_addr, src0_addr, src1_addr);
  ......
  asc_vf_call<add_vf>(dst_addr, src0_addr, tmp_addr);
  ......
}
```

2 VF(Vector Function)必须使用__simd_vf__修饰

```
__simd_vf__ inline void add_vf(__ubuf__ float* dst_addr,
   __ubuf__float* src0_addr, __ubuf__ float* src1_addr)
{
  ...
  add_vf_compute(dst_reg,  src0_addr, src1_addr);
  asc_dump_reg(dst_reg, 60000, 256);
  asc_store(dst_addr , dst_reg, mask);
  printf("simd vf add");
}
```


CANN

SIMT 调试调优接口 —— printf/assert

1 VF(Vector Function)必须使用__simt_vf__修饰

```
__simt_vf__ __launch_bounds__(THREAD_COUNT) inline void
simt_gather(__ubuf__ float* input, __gm__ uint32_t*
index,
     __gm__ float* output)
{
    int32_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    .......
    printf("simt vf gather: thread idx: %d.\n", idx);
    ......
}
```

2 业界编程体验一致的SIMT

```
__global__ __launch_bounds__(MAX_THREAD_COUNT) void
gather_custom(type_data* input,
    type_idx* index, type_data* gather_output,)
{
    int32_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    .......
    printf("gather_custom: thread idx: %d.\n", idx);
    ......
}

gather_custom<<<1, 1, dyn_ubuf_size, stream>>>(input,
index, gather_output);
```


CANN

C API 和C++基础API 调测接口对比

C++ 基础API 接口提供功能与C API接口功能一致，匹配不同开发者的习惯，部分映射关系如下：

类型	C++ 基础API	C API
printf	AscendC::printf	printf
assert	assert	assert
	ascendc_assert	asc_loadalign
dump	AscendC::DumpTensor	asc_dump_gm
		asc_dump_ubuf
		asc_dump_cbuf
		asc_dump_reg
time	AscendC::PrintfTimeStamp	asc_time_stamp

本章小结

- **介绍Ascend C算子调试调优接口能力**
  - ✓ AICore 调试调优接口: printf/DumpTensor/assert等
  - ✓ SIMD VF调试调优接口的能力
  - ✓ SIMT VF调试调优接口的能力

目录

Part 1 算子开发调试调优总览概述

Part 2 单算子、入图开发方式介绍

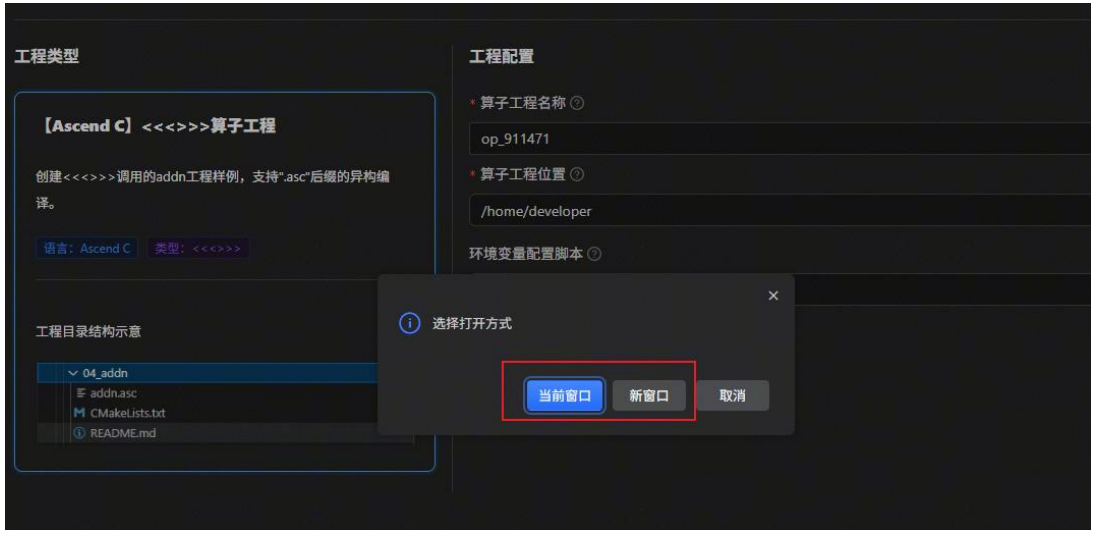
Part 3 调测接口使用方法

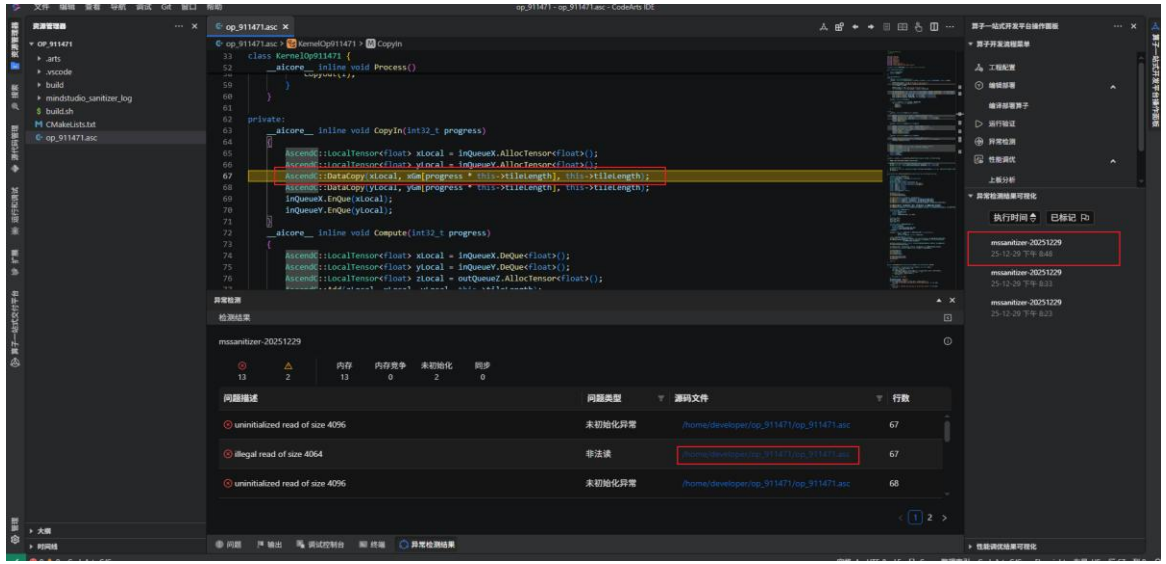
Part 4 配套工具的使用方法

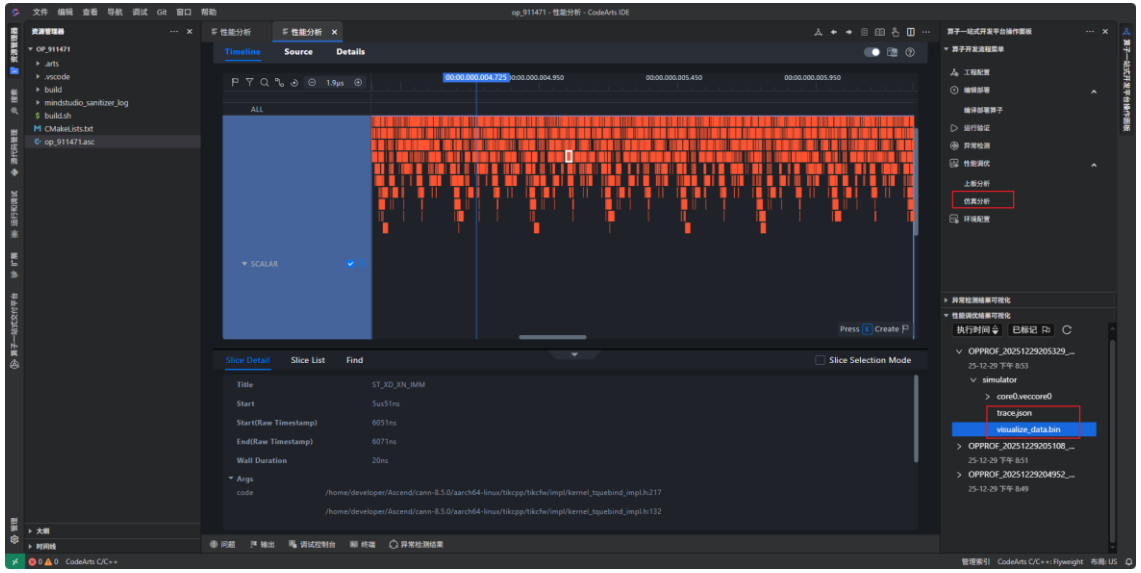
Ascend C算子调试调优工具能力

MS算子开发工具为了支持用户更好地**"看见"算子运行时昇腾AI处理器的状态**，支持在算子运行过程中，与AI Core进行交互，或者状态的记录。

VS code插件 | 算子一站式开发平台







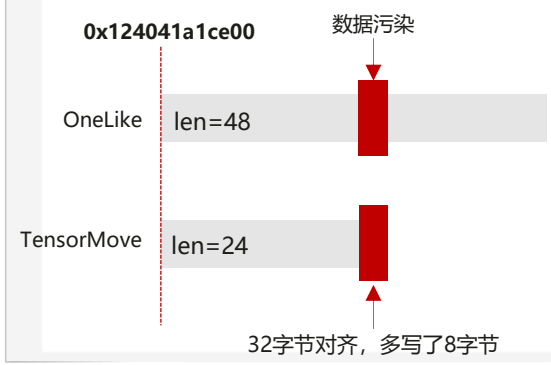
https://gitcode.com/cann

https://gitcode.com/org/cann/discussions/54

msSanitizer | 异常检测能力 快速定位内存问题场景

bisheng -x asc main.asc -npu-arch=dav-3510 -o main --sanitizer -g

内存检测



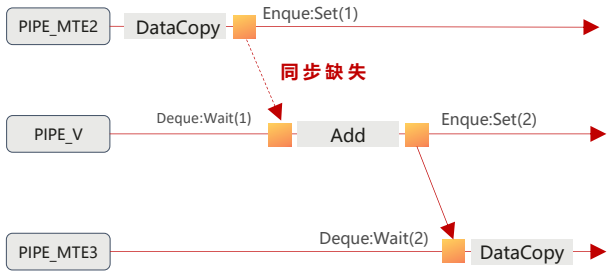

```
====== ERROR: illegal write of size 8
======     at 0x124041a1ce18 on GM
======     in block aiv(12)
======     code in TensorMove.cc:162
```

检测指令内存异常行为

精准定位异常代码行

mssanitizer -tool=memcheck ./main

竞争检测



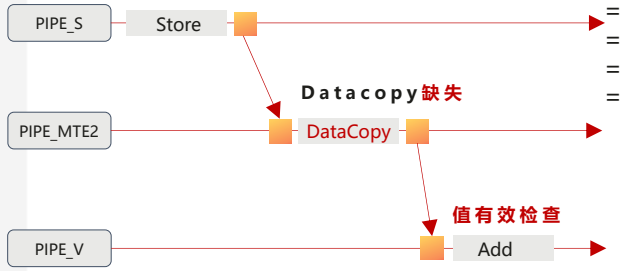

```
====== ERROR: Potential RAW
hazard detected at core0 in UB:
======     PIPE_MTE2 Read at
RAW()+0x0 in kernel.cpp:17
======     PIPE_V Write at
RAW()+0x0 in kernel.cpp:22
```

精准定位竞争内存位置

定界竞争引入位置辅助修复

mssanitizer -tool=racecheck ./main

未初始化检测



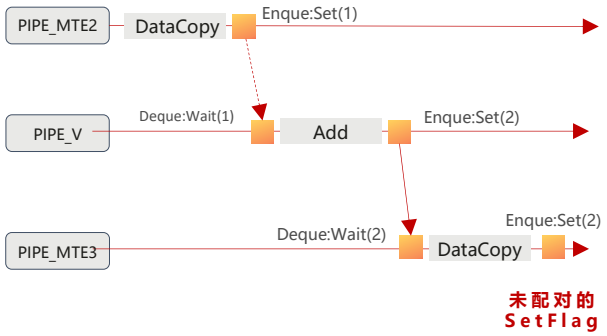

```
====== ERROR: uninitialized read of size 256
======     at 0x0 on UB
======     in block aiv(0)
======     code in kernel.cpp: 96
```

识别未初始化内存的读取

精准定位异常代码行

mssanitizer -tool=initcheck ./main

同步检测




```
====== WARNING: Unpaired
set_flag instructions detected:
====== from PIPE_V to PIPE_MTE3
in kernel
====== in block aiv(0) on device 0
====== code in pc current 0x2c94
(serialNo:25)
======     #0 kernel.cpp:17
```

识别未配对的SetFlag同步指令

避免影响后续算子同步行为

mssanitizer -tool=synccheck ./main

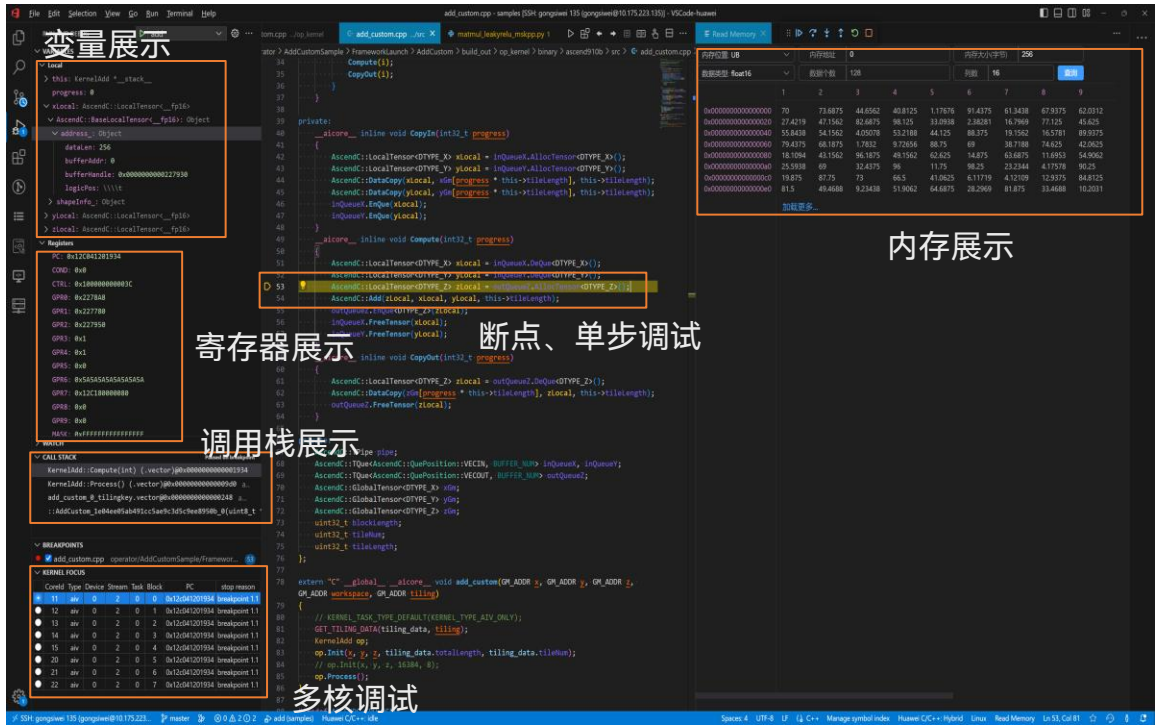
CANN

msDebug | 提供上板/仿真调试能力 高效定位算子异常

真实运行状态交互 灵活掌控程序行为

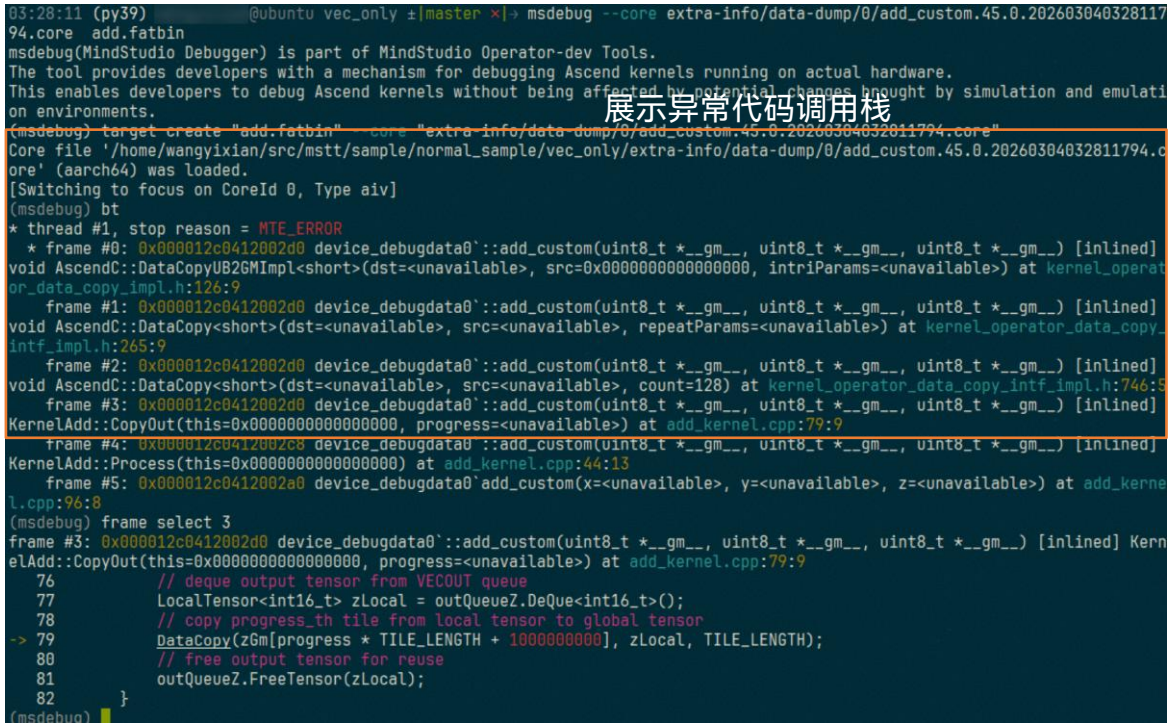
当算子功能异常时，通过单步跟踪程序执行，找到异常代码位置

bisheng -x asc main.asc –npu-arch=dav-3510 –o main –g
msdebug ./main



解析CoreDump文件回溯异常代码调用栈

展示算子异常时回溯异常代码调用栈



https://gitcode.com/cann

CANN

msProf | 算子调优数据的使用——7类性能指标数据

算子性能数据、block级指标直观平铺展示，总览算子性能。

msprof op ./main

msprof op --aic-metrics=Memory,PipeUtilization ./main

https://www.hiascend.com/document/detail/zh/mindstudio/830/GUI_baseddevelopmenttool/msascendinsightug/Insight_userguide_0034.html

交付件

OPPROF_{timestamp}_XXX

```
├── dump                        // 驱动等原始数据
├── OpBasicInfo.csv             // 算子基础信息，包含算子名称、block数量、运行频率和整体耗时等
├── ArithmeticUtilization.csv   // Cube/Vector计算类指令的耗时占比和数据量
├── L2Cache.csv                 // L2 Cache命中率
├── Memory. Csv                 // 内存读写带宽速率及通路带宽利用率
├── MemoryL0.csv                // L0A/L0B/L0C读写带宽速率
├── MemoryUB.csv                // UB读写带宽速率
├── PipeUtilization.csv         // 计算单元和搬运单元指令耗时和占比
├── ResourceConflictRatio.csv   //  资源冲突占比
├── visualize_data.bin          // 其他性能指标展示
```

Op Name	Op Type	Task Duration(us)	Block Dim	Mix Block Dim	Device Id	Pid	Current Freq	Rated Freq
_ZN7Catlass13k	cube	17.00034	20	NA	0	693832	1650	1650

block_id	sub_block	aic_time(us)	aic_total_cycles	aic_cube_time(us)	aic_cube_ratio	aic_scalar_time(us)	aic_scalar_ratio	aic_mte1_time(us)	aic_mte1_ratio	aic_mte2_time(us)	aic_mte2_ratio	aic_mte3_time(us)	aic_mte3_ratio	aic_mte3_active_bw(GB/s)
0	cube0	7.777576	12833	5.045455	0.648718	3.762424	0.483753	3.835757	0.493182	4.88	0.627445	0.000606	0.000078	1573.562622
1	cube0	7.763031	12809	5.024242	0.647201	3.93697	0.507143	3.791515	0.488407	4.671515	0.601764	0.000606	0.000078	1573.562622
2	cube0	7.895758	13028	5.020606	0.635861	3.68303	0.466457	3.842424	0.486644	5.070909	0.642232	0.001212	0.000154	786.781311
3	cube0	7.828485	12917	4.993939	0.637919	3.837576	0.490207	3.889697	0.496865	4.812121	0.614694	0.000606	0.000077	1573.562622

CANN

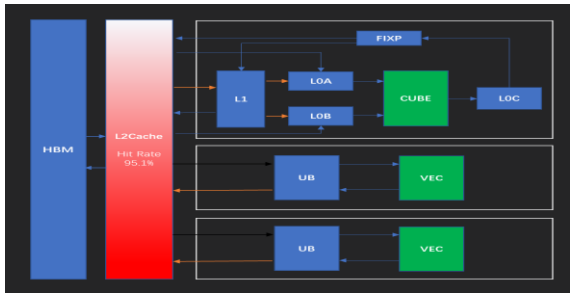
msProf | 多维度展示资源使用情况 高效识别瓶颈代码

核间负载图

直观展示算子运行时，每个核的负载均衡情况

msprof op --aic-metrics=Occupancy,Default ./main

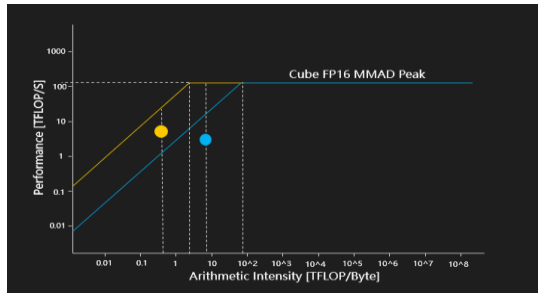
计算内存热力图



资源维度展示利用率，全局视角识别瓶颈

msprof op --aic-metrics=Memory,Default ./main

Roofline性能分析



理论维度展示优化空间，快速评估瓶颈优化收益

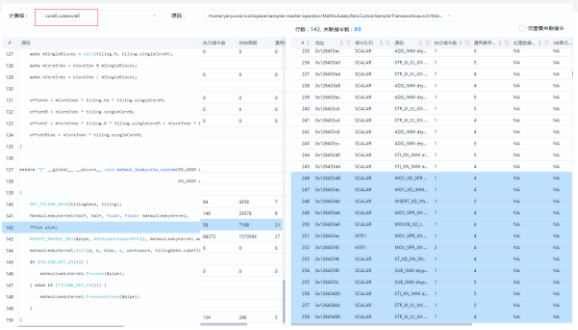
msprof op --aic-metrics=Roofline,Default ./main

指令流水图

指令维度展示时序关系，关联调用栈快速追踪瓶颈位置

msprof op simulator ./main

代码性能度量

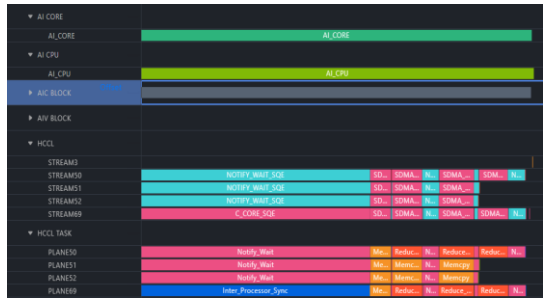


代码维度展示片段耗时，分析热点函数优化可行性

msprof op --aic-metrics=Source,Default ./main

msprof op simulator ./main

通算流水分析



提供通算融合算子的流水图、性能数据展示

msprof op ./main

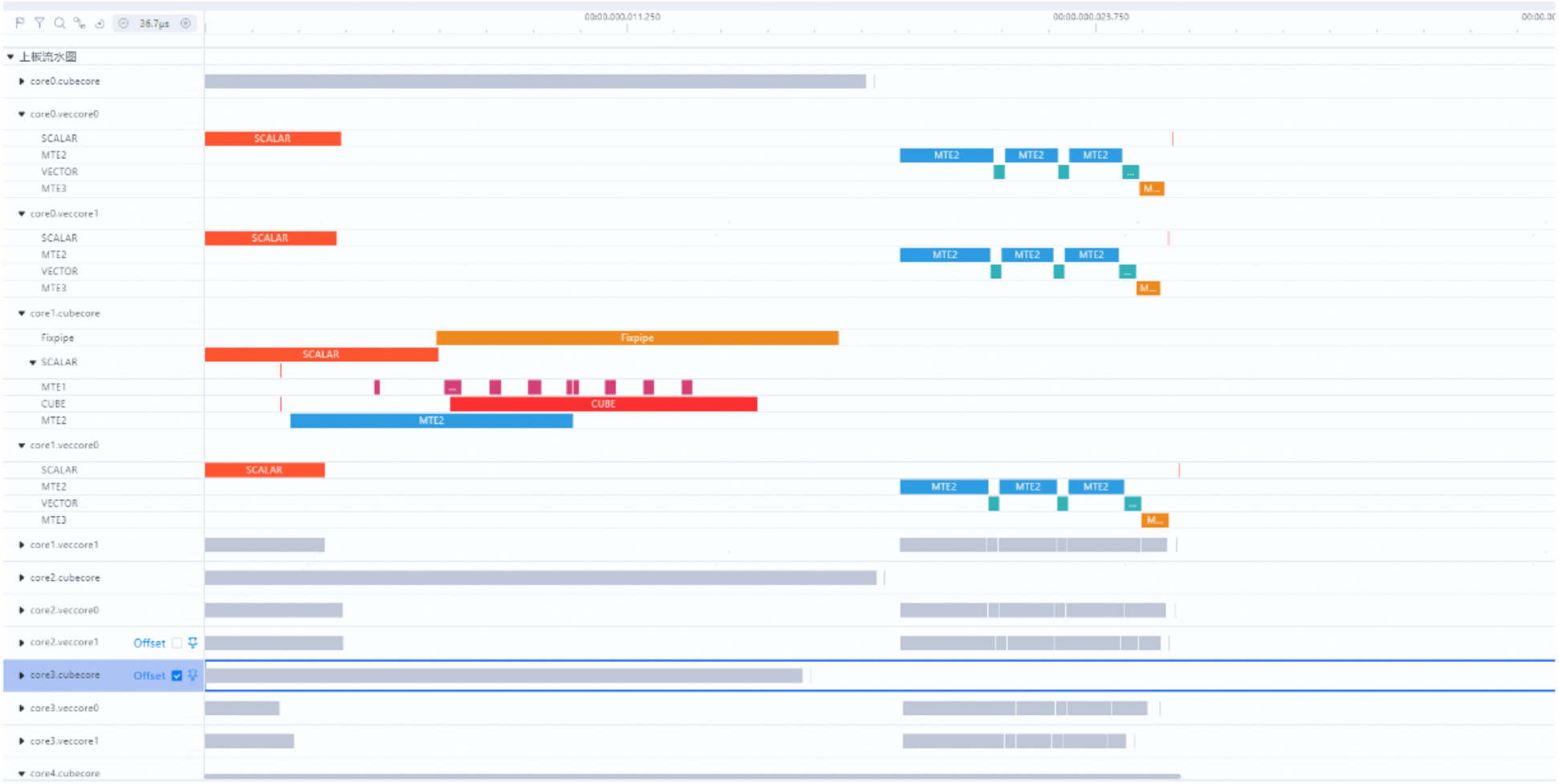
CANN

msProf | 算子调优数据的使用——Pipe流水图

主要采集算子AI CORE各个单元的活跃情况，分析算子执行的性能瓶颈

msprof op --aic-metrics=Timeline ./main

图 1 Pipe流水图



msProf | 算子调优数据的使用 —— SIMT VF 指令 Stall 分析

主要采集算子SIMT VF在执行时Stall的原因，用于分析SIMT VF性能瓶颈

msprof op --aic-metrics=PcSampling ./main

Data Manager

Import Data

D:\Download\自验证\热力图\...
D:\Download\自验证\热力图\OPPR...
D:\Download\自验证\热力图\...
D:\Download\自验证\热力图\OPPR...
D:\Download\自验证\热点图\...
D:\Download\自验证\热点图\OPPR...

Timeline | Source | Details

Core : ALL

Source : /home/wanghaopeng/workspace/operator/simt/kernel.cpp

#	Source	all Sampling(All Sa...	Stall Sampling(Not I...	GP...
8	`__simt_vf__ __aicore__ LAUNCH_BOUND(2048) inline void SimtCompute(__gm__ T *dst, __gm__ T *src0, __gm_`			
9	`{`	0%	0.00%	2
10	`int idx = Simt::GetThreadIdx<0>();`	1%	0.01%	5
11	`int blockIdx = Simt::GetBlockIdx();`			
12	`int tmp = 10;`			
13	`for (int i = idx; i < idx + 2000; ++i) {`	6%	0.06%	5
14	`auto val1 = src1[i];`	0%	6.44%	11
15	`auto val2 = src0[i];`	08%	17.44%	10
16	`auto val3 = val1 * val2 + tmp;`	98%	49.99%	9
17	`if (val3 % 4 == 0) {`	9%	0.09%	10
18	`val3 = val2 * tmp + val1;`			
19	`}`			
20	`dst[i] = val3;`	71%	21.14%	10
21	`}`			
22	`}`	0%	0.00%	0
23				
24	`template <typename T>`			
25	`__aicore__ inline void SimtComputeShell(GM_ADDR x, GM_ADDR y, GM_ADDR z)`			
26	`{`			
27	`__gm__ T *dst = (__gm__ T *)x;`			
28	`__gm__ T *src0 = (__gm__ T *)y;`			
29	`__gm__ T *src1 = (__gm__ T *)z;`			
30	`AscendC::MarkStampImpl<PIPE_V, 14>();`	0%	0.00%	0
31	`Simt::VF_CALL<SimtCompute<T>>(Simt::Dim3{512, 2, 2}, dst, src0, src1);`	0%	0.00%	0
32	`}`			
33				
34	`extern "C" __global__ __aicore__ void vec_add(GM_ADDR src0, GM_ADDR src1, GM_ADDR out, GM_ADDR gm_tili`			
35	`{`	0%	0.00%	0
36	`SimtComputeShell<uint32_t>(out, src0, src1);`	0%	0.00%	0
37	`}`	0%	0.00%	0
38				
39	`extern "C" void vec_add_do(uint32_t blockDim, void *l2ctrl, void *stream, uint8_t *src0, uint8_t *src1`			
40	`{`			
41	`vec_add<<<blockDim, l2ctrl, stream>>>(src0, src1, out, nullptr);`			
42	`}`			

Line : , Related Instructions Count : 0

Only Related Instructions

#	Address	Pipe	Source	Instructions Ex...	GPR Count	Stall Sampling(All Sa...	Stall Sampling(Not Is...
1	0x1000444000bc	VECTOR	VF_SIMT Xn, Xm, #in...	1	0	0.00%	0.00%
2	0x1000444000c4	SCALAR	MOVI	1	0	0.00%	0.00%
3	0x1000444000c8	SCALAR	DCCI	1	0	0.00%	0.00%
4	0x1000444000cc	SCALAR	END	1	0	0.00%	0.00%
5	0x1000444000d0	VECTOR	S2R	2	2	0.00%	0.00%
6	0x1000444000d8	VECTOR	S2R	2	3	0.00%	0.00%
7	0x1000444000e0	VECTOR	SHFI	2	3	0.01%	0.01%
8	0x1000444000e8	VECTOR	IADD.i	2	4	0.00%	0.00%
9	0x1000444000f8	VECTOR	IADD.i	2	5	0.00%	0.00%
10	0x100044400108	VECTOR	START_DVG	2	3	0.01%	0.01%
11	0x100044400110	VECTOR	MOVI	2	4	0.00%	0.00%
12	0x100044400118	VECTOR	FMUL.i	2040	6	0.04%	0.03%
13	0x100044400128	VECTOR	FMUL.i	2040	8	0.07%	0.06%
14	0x100044400138	VECTOR	FMUL.i	2040	10	0.05%	0.03%
15	0x100044400148	VECTOR	LDG	2040	11	6.65%	6.40%
16	0x100044400150	VECTOR	LDG	2040	10	17.52%	17.38%
17	0x100044400158	VECTOR	FMUL.i	2040	9	49.37%	49.99%
18	0x100044400168	VECTOR	FMUL.i	2040	10	0.01%	0.00%
19	0x100044400178	VECTOR	Lop3	2040	9	0.01%	0.01%
20	0x100044400188	VECTOR	ISETP.i	2040	7	0.03%	0.03%
21	0x100044400198	VECTOR	SEL	2040	7	0.05%	0.04%
22	0x1000444001a0	VECTOR	ISETP	2040	5	0.05%	0.03%
23	0x1000444001a8	VECTOR	IADD.i	2040	4	0.03%	0.03%
24	0x1000444001b8	VECTOR	STG	2040	3	2.35%	2.23%
25	0x1000444001c0	VECTOR	BRANCH	2040	0	19.02%	18.88%
26	0x1000444001c8	VECTOR	END_DVG	0	0	4.73%	4.83%
27	0x1000444001d0	VECTOR	END	3	0	0.00%	0.00%
28	0x1000444001d8	VECTOR	LDG	0	1	0.00%	0.00%
29	0x100044400000	SCALAR	MOV_SPR2X	1	0	0.00%	0.00%
30	0x100044400004	SCALAR	MOVK	1	0	0.00%	0.00%
31	0x100044400008	SCALAR	DC_PRELOADI	1	0	0.00%	0.00%
32	0x10004440000c	SCALAR	MOVI	1	0	0.00%	0.00%
33	0x100044400010	SCALAR	MOVK	1	0	0.00%	0.00%
34	0x100044400014	SCALAR	MOV_SPR2X	1	0	0.00%	0.00%
35	0x100044400018	SCALAR	MOV_SPR2X	1	0	0.00%	0.00%

CANN

本章小结

- **介绍Ascend C 算子工具**
  - ✓ 算子编码：Vscode 插件；
  - ✓ 上板调试：msDebug；
  - ✓ 上板异常：msSantizer；
  - ✓ 性能调试：msProf；
  - ✓ 可视化工具：MindInsight

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN